DAT321/DIT847: Software Quality

Welcome to the examination for the *Software Quality* course. Each question has a number of points assigned shown in the square brackets. When the question is broken down into smaller sub-questions the part of the points for that specific sub-questions are also shown as following:

1. *[10 pts].*
    a. *[2 pts]*
    b. *[8 pts]*

The percentage of points and the corresponding grade is presented below (100 points in total):

% of points	DAT321	DIT847
[0, 50%)	U	U
[50%, 65%)	3	G
[65%, 85%)	4	G
[85%, 100%]	5	VG

Instructions about how to submit this exam are found in the Assignments section of Canvas (under the Written Exam Submission page). You must submit a PDF file with your answers. Make sure that you label your answers with the question numbers so we can identify which question your answer is referring to. You can choose any editor to write your answers.

Important: You must write clear, readable, understandable and unambiguous answers. An advice is to refer to the provided software product context used in this exam. Whenever possible refer to modules, features and information about the software product or software development process described in the context.

The questions in this exam refer to the **ISO 25010:2011** that categorises internal and external software quality attributes into eight characteristics, as well as the quality in use characteristics.

Questions about the exam contact:
Francisco Gomes, tel. 031 772 69 51, francisco.gomes@cse.gu.se

The exam review will be done via Zoom scheduled for:
Date: 2021-01-22 between 09:00 – 10:30.
Zoom Link: https://chalmers.zoom.us/j/62891716138
Password: 317535

The questions in this exam are related to the context below. Note that you **must** justify your exam answers with i) the theory and terminology from software quality and ii) their connections to the elements in this context (e.g., teams, product, processes, architecture, customers, etc.).

You are hired by a media streaming company Stream-Co that offers the most popular **media streaming services on the Internet.** The software product for this exam is a real-time streaming platform where end-users can watch several movies and series. Examples of similar software products are Netflix, HBO Nordic, HBO Max, Prime Video, Disney+, etc.

Features, business and customers:
In order to access the software, users must first login to access the media catalogue made available by Stream-Co. When first creating an account, users must choose a subscription plan with different payment models (e.g., monthly, yearly) where the user provides credit card or banking information to Stream-Co. Upon successful login, the UI Platform (**Module A**) shows the media available to the user so that they can begin to watch their favourite media. The UI Platform is installed in the user's client side and is built to allow access from various hardware devices, such as a laptop's browser, mobile phones tablets or even directly via Smart TVs.

A Subscription platform (M**odule D**) handles all storage and access to user's account information such as e-mail, name, home address, phone number. The billing of accounts is handled by another software provided by a partner company named Bank-Co. In order to ensure communication, Stream-Co uses a Billing platform (**Module E**) able to communicate with the software developed by Bank-Co and bill its user.

Stream-Co also uses a Streaming platform (**Module B**) from which the media data is sent over the internet to the user's screens. Since users have different quality of internet connections, the Streaming platform automatically adapts the quality of the video to better match the user's connection bandwidth. In the background, a Recommender Platform (**Module C**) monitors the streaming data and the users' choices in order to learn and understand viewing patterns of the user and recommend new series and movies by using different machine learning algorithms.

Design and architecture:
The software product uses a typical Client-server architecture (Figure 1), and users watch live content using an Internet connection. Each module's responsibilities are listed below; they are connected to the features of the system listed above. These modules exchange information and also depend on each other for the streaming service to function properly.

Modules A is deployed in the client side of the architecture, whereas Modules B, C and D are deployed in the server side of the architecture. Module F is its own platform developed and deployed by Bank-Co. Our only responsibility is to ensure that the interfaces between the Billing and Subscription platforms can communicate to know which payments were successful.

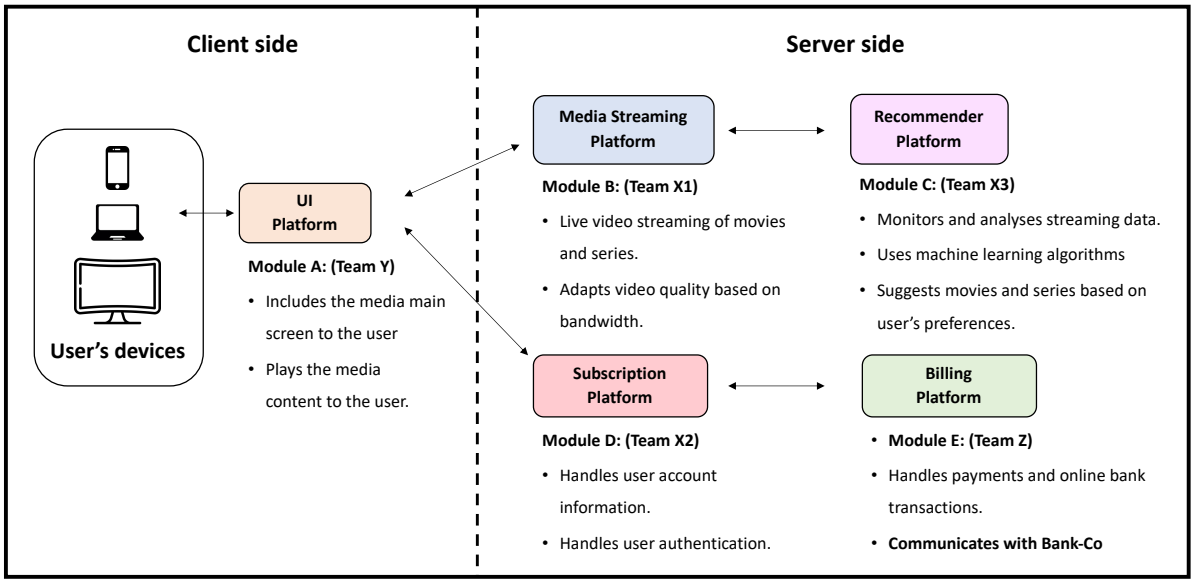

Figure 1 – Summary of Modules and Features of the Stream-Co Product

Development process and teams:

There are three teams in Stream-Co. Team X is a team of 30 engineers divided into smaller teams of roughly equal size that work with the server side of the service (Modules B, C and D). Teams X1 and X2 work on the Streaming and Subscription platforms, respectively. They are composed of a mix between junior and senior architects, developers and testers experienced with agile methods. Team X3 is new to Stream-Co and is composed of junior developers, two senior software engineers and one data scientist experienced with machine learning. Most members of Team X3 have not worked together and did not use agile methods before.

Team Y is composed of 10 engineers working in the client side of the service (Module A) and is composed of developers, testers and one user experience (UX) designer. Finally, Team Z is a small team of 2 engineers responsible for maintaining the communication with Bank-Co (Module E). All members of Teams Y and Z have worked with agile development before.

Each team has their own isolated development environment using version control. However, only Team X3 is **not** using continuous integration (CI), such that the necessary tool chain for CI is not installed in their development environments.

Software Product – Future plans:

Stream-Co is aiming to expand and include its own social network platform in the application. Via this social network, users that watch the same videos can share messages and add each other as contacts. In order to save time and costs with software development, the Social Network Platform (Module F) aims to reuse the Subscription and Recommender modules. Therefore, Stream-Co will soon hire engineers for Team S to develop Module F.

1. **[25 pts]** Using the context above, answer the following questions:

   a. [15 pts] Choose **three** software product quality's characteristics, **and, for each,** provide examples of why they are relevant to the software product at Stream-Co. When justifying your choices, you **must** use the features and modules listed in Figure 1 as sources for your examples.

Any three examples of the 8 below would work. The key point here is to connect the **description** of the software product quality characteristic (i.e., only Internal or External allowed, not quality in use) to the context above, specially the modules. The amount of points depends on the clarity of the justification behind the relevance. Ideally, for full points you should exemplify one Modules or Teams of the context provided.

(Each description of relevance: 2-3 points, Using Modules or Teams: 2-3 points).

(There should be 3 at least … if there are more, we will look at the totality of what you wrote. For instance, 4 examples in which only 3 of them are correct will not award you full points since there are still wrong descriptions provided.)

1. Security: This quality characteristic is particularly relevant since there are banking transactions made in Module E, hence the application needs to ensure the right access level to its users. Alternatively, this quality characteristic is relevant because users need to do login and via Module A (or Module D) to access the streaming features.

2. Compatibility: This is relevant for the development of the Billing Platform since the product needs to share information between other software applications such as the banking interface in Module E. So the software should properly share the user information with other software products.

3. Portability: This is relevant for the product because it must run on different types of hardware devices (TVs, laptops, tablets, phones) and the UI Platform (Module A) must be able to function properly in those various devices.

4. Functional suitability: This is relevant since the users expect to receive quality streaming service, so Modules A, B and D are connected to this characteristic. Alternatively, for specific functionality one can also cite Modules C if mentioning the video recommendations or Module E if connected to the automatic billing functionality.

5. Performance efficiency: This is particularly relevant for the product since the videos should adapt to the resource capacity of the user (i.e., the network bandwidth). Otherwise, users risk watching low quality video on high quality connections or for low quality connection the streaming can ruin the viewing experience via constant buffering. So this is highly connected to Module B.

6. Maintainability: This is relevant due to the updates needed in the software to introduce the new social network platform (Module F) and avoid issues with the working modules of the system. Alternatively, updates in the recommender system (Module C) due to changes or updates in the machine learning should also not introduce faults or design issues in the product.

7. Reliability: This is relevant to ensure uptime of the streaming service so that it is available to users at any time of the day and anywhere in the world. Particularly since the paid plans are based on time (daily, monthly), the users must have access to the streaming service as often as possible. So, this is highly connected to Modules B, C or E.

8. Usability: This is relevant for adapting the product to cope with unique affordances in different hardware devices that offer different interfaces (e.g., buttons, screen resolutions). This is highly connected with Module A.

b. [10 pts] Choose **two** quality in use characteristics and explain which modules or features we could use to measure those chosen characteristics.

Any two examples of the 5 below would work (other examples would also be acceptable if they comply with the definition of the characteristic). In addition to connecting with the description, you must provide a context of use of the product. Examples of contexts are searching for a series in the browser, or using the video player in the mobile phone, or watching until the end the videos that were recommended. Points distribution would be, for each characteristic: (i) mentioning relevance in connection to a module (1-2 points), (ii) mentioning context in use (2-3 points). Its likely that most Quality in Use measures will happen at the UI since they are related to the user.

1. Satisfaction: We could use Module A to measure satisfaction by measuring how many times, over a fixed period (e.g., month) the user returns to the interface to watch their videos. Another example: We could use Module D or Module E to measure how much time is passed until users cancel their paid subscriptions. We could measure satisfaction by including forms in Module A to ask users to rate their satisfaction with the web browser interface.

2. Effectiveness: We can measure effectiveness by counting the number of times the user returns to the main screen (Module A) after navigating the mobile interface. Alternatively, by counting the number of times the user exits the main screen in the web browser (Module A) without watching any video.

3. Efficiency: We can measure efficiency by counting the number of videos played in each different device (Module A) during a day of paid subscription (i.e., how much money the customer actually pays in relation to how much time they watch). We can measure efficiency by counting the amount of battery consumed when the user watch videos in their phone (Module A).

4. Freedom of risk: We can measure freedom of risk by analysing the impact of the costs of paid subscription (Module D) and the user's monthly expenses to see the financial risks that the service would pose to our user's finances. Another example is to measure the risk that extended hours watching too many videos (Module A) on, e.g., a computer screen would pose to the user's health. Alternatively, measuring the extent in which users watch videos in their phones (Module A) while driving.

5. Context of coverage: We can measure it by measuring the time spent on videos when users are on a small screen (Module A) with low bandwidth network (Module B). Another example is measuring which devices' screens are used more often (Module A) and how that correlates to different subscription plans (Module D)

2. **[15 pts]** Stream-Co wants to foster sustainability in ICT. Using the Sustainability Awareness Framework, you must (i) choose or suggest **three product features** and explain the (ii) immediate, (iii) enabling and (iv) structural effects of those features. You must also indicate the **corresponding dimension** in which those effects occur.

   For this question you are free to choose to modify an existing feature (from Figure 1) or create a new feature. We refer to feature as a functionality in our software product. Make sure to clearly distinguish between the immediate, enabling and structural effects in your answer.

   Similar to question 1, there are many right answers here. Each example is 7.5 points. The key is to provide a clear connection between the immediate, enabling and structural effects (5-6 points for the quality of examples and connection), and the appropriate choice of feature and corresponding dimensions (1-2 points).

   Suggested feature: Promote videos or series (e.g., documentaries) about current social issues on equality (Module A and C). Immediate effect: Bring awareness about current societal problems (social dimension), enabling effect: Increase visibility of minorities and the current issues they face (social dimension); Structural effect: Encourage changes to improve equality (social dimension).

   Suggested feature: Use the service's social network to connect users with similar interests (Module F). Immediate effect: Create communities with shared interests (individual dimension); Enabling effect: Promote social engagement among users and different communities (social dimension); Structural effect: Risk of creating closed communities (silos or bubbles) among the users (social dimension).

   Suggested feature: Provide feedback and analytics on user's screen time in relation to the hours and days of the week (Modules A and D). Immediate effect: Bring awareness to user's viewing habits (Individual dimension). Enabling effect: Encourage user to spend less time viewing the

screen and seek other activities (individual dimension). Structural effect: Less time using the service can lead to less energy consumption from the devices (environmental dimension).

3. **[15 pts]** Considering any of the features or modules in Figure 1, choose three different levels of testing and, for each level chosen, provide at least two examples of test cases.

This means that each test case description accounts for 2.5 points (two test cases for each of the three levels of testing – 6 examples in total). The levels of testing are: unit, integration, system or acceptance.

For each test case, you get points if you include (i) correct reasoning that the test case verifies the feature at the right level of testing (1.5 points), and (ii) comprehensible description of the test case as a scenario with conditions and expected output (1 point).

A few examples below:

Chosen feature: Login to the Streaming service
For Unit-level: an example of a test case is to verify the code class that checks whether a valid username and password provided matches the user information stored in the database. **Integration-level:** a test case is to verify whether the user authentication information sent by Module A can be properly received and handled by Module D which then returns the result of the authentication back to Module A. **System-level:** a test case is to verify whether users can login at the same time in multiple sessions (i.e., opening several tabs in different devices); **Acceptance-level:** a test case is to verify whether users have access to all navigation features after a successful login. Alternatively, we can also verify whether users with invalid login are indeed not able to access any features from the service.

Chosen feature: Select and play videos
Unit-level: a test case to verify whether the video file can be properly loaded and played from beginning to end. **Integration level:** a test case to verify whether the video can be sent from the Streaming platform (Module B) to the UI (Module A) by controlling the bandwidth. **System-level:** a test case to verify whether the player loads and plays a video until the end with adjusted resolution on different devices from Module A. **Acceptance-level:** A test case to verify whether the user can properly pause, rewind or forward the video while using the interface of a specific device (tablet or web browser).

4. **[15 pts]** Considering the decision to introduce Continuous Integration to Team X3, answer the following questions:

   a. [5 pts] What is the necessary development toolchain that Team X3 needs to have in order to properly use Continuous Integration.

Here, we award points per tool and justification of the tool (1-2 points each) Version control

does not count because they all have version control according to the description. So, as a minimal, (i) Team X3 would need an automation server like Jenkins or Travis CI in order to schedule builds of the commits sent by the engineers, and (ii) an automation tool such as Graddle or Maven to automatically package and create a working version of the software with the latest code changes.

b. [10 pts] List **two activities** you would do or teach to Team X3 before introducing CI. Justify why those activities would be relevant to introduce CI.

Here you would receive 5 points for the clarity and correctness of your activity. Correctness is analyzed if the activity is related to the main requirements of adopting CI: (i) adopting and using configuration management, (ii) developing team practices for continuously testing, and (ii) making small but frequent changes in the software product. For this context, using version control software (git, svn, etc.) and instrumenting automated builds and the entire tool chain is not accounted for points because they were mentioned in the previous question (4.a).

These activities must be well described (2 points) and properly connected to the context (3 points). Example of answers are:

The context states that Team X3 is not familiar with agile, so they require training in making smaller and more frequent changes as well as handling unexpected software changes. For instance, they should create the habits of committing regularly (e.g., at least once per day) in order to test the application to trace the small iterations of the product.

Continuous testing should be taught since the team must actively create and maintain a repository with automated tests that execute every time a commit is submitted. For instance, changes in Module C should be tested automatically before integrating it with the other Modules.

5. **[20 pts]** Considering internal/external quality measures and quality attributes, answer the following questions:

a. [5 pts] An analysis of technical debt revealed that all modules need refactoring and have the same debt, such that restoring the maintainability of each module would take 10 days. You only have resources to choose one module to reduce this technical debt. **Which module** would you choose and **why**?

Here, the points are only awarded to choosing the correct module (2 points) and providing the correct reason (3 points). The correct modules are C or D since they are going to be reused soon in the creation of Module F (social network), there modifications on either of those modules would benefit Module F. You could also mention the fact that Module C is even riskier to do changes since Team X3 is not very experienced with good practice which entails changes.

Suggesting other modules will not award you points.

b. [5 pts] You were asked to design a two-dimensional chart (x-axis and y-axis) to visualise the internal quality of the different modules in Figure 1. Which two quality measures would you show in this chart? Which quality attributes would be easier to understand using your proposed visualisation?

Note that the question says Internal Quality, so metrics derived from software execution are not applicable (test results, network latency, resource usage, number of players joining or leaving). The metrics must be connected to the software artefacts. Given the focus on architecture, one of the axes (e.g., y-axis) should include Henry & Kafura complexity, Fan-in or fan-out, number of methods calls to other modules (choice or right measure = 1 point), or other measure that makes it easier to understand modularity or coupling (2 points), since each team is responsible for a module, so this would allow teams to have a shared view of the module dependencies (1 point). The second axis can be a complementary metric with many options. The important part is providing the right justification for the combination chosen. For instance (2 points):

- number of changes or commits, allows us to see which modules change more often.
- size of the module in lines of code, allows us to see which modules are bigger or smaller to allocate refactoring resources.
- cyclomatic complexity, allows us to see which modules are easier or harder to test.
- number of faults (not failures!), allows us to see which modules often require more debugging effort

c. [10 pts] Provide **two examples** of external quality measures for the software product. For each example, you must explain which module or feature from Figure 1 they would be associated with.

Note that the question says External Quality, so metrics must be derived from software execution. You get points by suggesting a correct metric (2 points) and providing a correct reasoning about the choice (3 points).

Test executions is one example if you can justify based on a specific module. For instance, the number of failures in a specific module to verify whether the subscriptions are properly created and stored (Module D). However, the number of failures can only be mentioned once, otherwise it's a redundant answer.

Other examples are: Measure the uptime of the streaming service on different load configurations (Module B). Measure the stability of the video quality under different bandwidth configurations (Module B). Measure the number of successful bank transactions for each subscription plan (Module E).

6. **[10 pts]** The lead tester in Team X1 wants to improve the quality of all server-side modules by comparing two new code reviewing techniques named Technique A and Technique B. The goal is identifying which technique is better at finding more code smells. She then asks

all junior and senior developers to use both techniques and measures the number of code smells found in the code. This process is repeated for Modules B and C. An example of the data collected during the experiment is shown below:

Engineer ID	Experience	Module	Technique	Number of Code Smells
P1	Senior	Module C	Technique A	8
P1	Senior	Module C	Technique B	7
P1	Senior	Module B	Technique A	15
P1	Senior	Module B	Technique B	20
P2	Junior	Module C	Technique A	4
P2	Junior	Module C	Technique B	8
P2	Junior	Module B	Technique A	10
P2	Junior	Module B	Technique B	12
…				

Write down the **mathematical model definition** for this experiment using **any** variable names and priors of your choice. State the ontological and epistemological reasons for your likelihood. Remember to clearly state and justify the choices and assumptions regarding your model.

(Up to 3 point for mentioning:)

In order to compare techniques, we must compare the posterior distribution to see which technique is more likely to reveal more faults. We need a GLM (General Linear Model) to predict the number of faults. The number of faults can be affected by the choice of technique, the experience/seniority of the tester, the module being used with the technique. There can be variations per participant, so we need varying intercepts.

(7 points based on the completeness and correctness of the Model)

The model below yields a posterior that allows us to predict number of faults based on different predictors:

$$faults \sim Poission(\lambda_i)$$
$$\log(\lambda_i) = \alpha + \beta_t * Technique + \beta_e * Experience + \beta_m * Module + \alpha_{Subject[i]}$$
$$\alpha \sim Normal(0, 5)$$
$$\beta_t, \beta_e, \beta_m \sim Normal(0,1)$$

For the hyperparameters/priors of the varying intercepts:

$$\alpha_{Subject} \sim Normal(\mu_s, \sigma_s)$$
$$\mu_s \sim Normal(0,1)$$
$$\sigma_s \sim Cauchy(0,2)$$